

Through The Ages

If you have documents to worry about, you should also worry about tracking changes made to them

Nimish Chandiramani

Ever opened an old, oft-edited Word document and wondered what it looked like last week? What about the time you made your own changes, only to find that someone else had overwritten the file with their own version? In the digital chaos we all live in, keeping track of your files—the changes made to them and the person making those changes—is something we all desire, but few actually achieve.

It's All About Control

To address all these needs and more, we turn to a system that software developers have been using for ages—version control. When you're working in teams where more than one person needs to edit the same code at a time, you really can't afford your work to be wiped out by a wayward team member, can you? Version control systems also track every single change made to a file, so if you want to see how the file started out and evolved, you can. The system also gives you a new layer of security—the ability to control access to your documents.

If you're worried about radical changes to the way you work, fret not. A version control system will fit in quite nicely with the way your company's files are already organised. Presumably, you have a central file server where all your official documents are stored—this is *all* the infrastructure you need.

How It Works

The best analogy for a version control system is a public library—the term we'll be using is *repos-*

itory. Library members (your employees) hunt down the book they want and go to the librarian (the version control software). The librarian then checks their library card, and makes sure that the book in question is indeed available to the member, before letting them check it out. The similarity takes a slight deviation here—when someone returns a file, it will contain changes, and the person checking it in will have to leave a little note indicating the changes made. The version control software then records what changes were made to the file (it's all binary data ultimately, so this isn't much of a problem) and stores the new version.

But what if more than one person wants the file? There are three ways that this software can handle the situation: the first is the exclusive check-out, where it denies access to anyone who tries to edit the file once it's been checked out. This is by far the easiest way to avoid any trouble, and is recommended for most scenarios. However, you might encounter situations where two people need to work on different parts of the file at the same time, and waiting for the file to be available just isn't an option. If there's a possibility of this happening often, then you'll need to opt for a setup where a file can be checked out by multiple users, and all changes made by them will then be merged into the document. If the same parts of the file have been edited by more than one person, the software asks for your intervention to determine whose change to accept or reject.

All this happens in a centralised system; for teams spread out over many cities or countries, a concept called *Distributed Version Control* comes into play. In such a case, everyone has the full repository, and changes to any document are propagated to everyone. This way, everyone



Illustration Chaitanya Surpur